Московский физико-технический институт
Высшая школа программной инженерии

Алгоритмы
3 семестр, 2026/2027 уч. год

Лекция 3–4
Суффиксный автомат

Лектор: **Кулапин Артур**
Автор: Ivan Gerunov

1. Суффиксный автомат

В прошлых лекциях мы заранее знали, *что* искать: был паттерн (или набор паттернов), по нему мы строили префикс-функцию или бор, а потом прогоняли через это текст. Теперь развернём задачу: зафиксируем текст S и построим по нему структуру, которая умеет отвечать на вопросы про *все* его подстроки сразу — например, «является ли P подстрокой S?» за $O(|P|)$, не читая S заново.

Самое простое решение — засунуть в бор все суффиксы S: любая подстрока — префикс какого-то суффикса, значит, читается в таком боре. Но в нём $O(|S|^2)$ вершин (для строки из разных букв все суффиксы ветвятся сразу из корня). Суффиксный автомат делает то же самое, но склеивает «одинаково ведущие себя» вершины, и в итоге у него $O(|S|)$ состояний и рёбер, а строится он за линию. Чтобы понять, *что* именно можно склеивать, нужна теорема Майхилла-Нероуда.

1.1 Правый контекст

Определение 1 *(Правый контекст). Правым контекстом слова x относительно языка L называют множество*

$$R_L(x) = \{w \mid xw \in L\}$$

То есть это все способы «дописать» x справа так, чтобы получилось слово из языка.

Пример:

$L = \{a, b, ab, aba, ba\}$

$R_L(ab) = \{\varepsilon, a\}$ (сами ab и aba лежат в L)

$R_L(aba) = \{\varepsilon\}$

$R_L(abab) = \varnothing$ (как $abab$ ни продолжай, в L не попадёшь)

$R_L(b) = \{\varepsilon, a\}$

Определение 2 *(Эквивалентность относительно языка). Два слова x и y эквивалентны относительно языка L (обозначение $x \underset{L}{\sim} y$), если $R_L(x) = R_L(y)$.*

Проще говоря, с точки зрения языка L эти слова неотличимы: что к одному ни припиши, к другому можно приписать то же самое с тем же результатом.

В примере выше:

$ab \underset{L}{\sim} b$

$aba \underset{L}{\sim} ba$

Теорема 1 *(Майхилла-Нероуда). Пусть у отношения $\underset{L}{\sim}$ конечное число классов эквивалентности, равное k. Тогда*

1. *язык L автоматный, и в любом ДКА, распознающем L, не меньше k состояний;*
2. *существует ДКА, распознающий L, ровно с k состояниями, где каждое состояние отвечает своему классу эквивалентности.*

Полное доказательство будет в курсе дискретной математики, здесь — идея, её достаточно.

1. Автоматность следует из пункта 2: там автомат строится явно. Докажем оценку. Пусть у ДКА меньше k состояний. Возьмём по одному слову из каждого класса — слов k, состояний меньше, значит, два слова x и y из разных классов приведут автомат в одно и то же состояние.

Но дальше автомат не помнит, как он туда пришёл: для любого w слова xw и yw тоже закончатся в одном состоянии и будут одновременно приняты или отвергнуты. Значит, $R_L(x) = R_L(y)$ — противоречие.

2. Сделаем состояниями сами классы: из класса $[x]$ по букве c идём в класс $[xc]$, начальное состояние — $[\varepsilon]$, принимающие — классы, у которых $\varepsilon \in R_L(x)$ (то есть $x \in L$). Переход не зависит от выбора x внутри класса: если $x \underset{L}{\sim} y$, то $R_L(xc) = \{w \mid cw \in R_L(x)\} = \{w \mid cw \in R_L(y)\} = R_L(yc)$. Прочитав слово x, автомат оказывается в $[x]$ и принимает его ровно тогда, когда $x \in L$.

Примечание. Главная мысль, которую стоит отсюда унести: **состояние автомата — это правый контекст**. Всё, что автомату нужно помнить о прочитанном префиксе, — какие продолжения приведут к успеху. Два слова с одинаковым правым контекстом можно смело склеить в одно состояние, а с разными — нельзя. Это ровно то, чего мы хотим от сжатого бора суффиксов.

1.2 Суфф. автомат

Определение 1 *(Суффиксный автомат). Суффиксным автоматом строки S называют минимальный по числу состояний ДКА, распознающий язык суффиксов S.*

По теореме Майхилла-Нероуда его состояния — это в точности классы эквивалентности по этому языку. Дальше вместо R_L и $\underset{L}{\sim}$ будем писать R_S и $\underset{S}{\sim}$:

$$R_S(x) = \{w \mid xw \text{ — суффикс } S\}$$

Посмотрим, какие вообще бывают классы. Пусть X — не подстрока S. Тогда Xw тем более не подстрока S, а значит и не суффикс, при любом w. То есть

$R_S(X) = \varnothing$

Итак, все не-подстроки S лежат в одном классе — «мёртвом» состоянии, из которого уже никогда не попасть в принимающее. Его мы рисовать не будем: суффиксный автомат — это *частичный* ДКА (как в прошлой лекции), и отсутствие ребра как раз означает «ушли в мёртвое состояние». Поэтому **далее речь идёт только о подстроках S**, а состояний в автомате столько же, сколько классов подстрок.

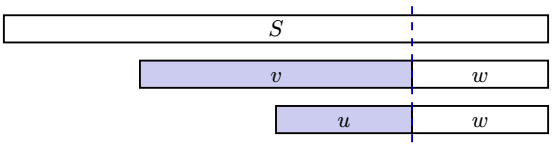
Определение 2 *(Обозначение класса). Класс эквивалентности подстроки u обозначим $[u]_S$ (или просто $[u]$, если понятно, о какой строке речь). Так как состояния — это и есть классы, то $[u]_S$ — это одновременно и состояние автомата, в которое мы придём, прочитав u из начального состояния.*

Важно: в прошлой лекции $[u]$ для вершины бора означало слово *на пути до неё. Здесь наоборот — $[u]$ это* состояние *(класс), в котором лежит слово u. Слов в одном классе может быть много.*

Из определения сразу получается ещё один взгляд на состояние: $R_S(u)$ — это ровно набор слов, по которым можно дойти из состояния $[u]$ до терминального. То есть **правый контекст состояния — это набор путей из него до терминала**: два слова эквивалентны, если из их вершин «видно одно и то же будущее». Терминальные состояния — это те, у которых $\varepsilon \in R_S$, то есть классы, содержащие суффиксы S.

Утверждение 1. *Пусть $u \underset{S}{\sim} v$. Тогда либо u — суффикс v, либо v — суффикс u.*

Действительно, u и v — подстроки, поэтому $R_S(u) = R_S(v) \neq \varnothing$. Возьмём любое w из этого множества: uw и vw — оба суффиксы S. Значит, u и v заканчиваются в одном и том же месте — ровно за $|w|$ символов до конца S, и более короткое из них — суффикс более длинного.



Раз все слова класса — суффиксы друг друга, в классе есть самое длинное слово, и все остальные — его суффиксы. Введём обозначения.

Определение 3 *(longest и len).* $\text{longest}(v)$ *— самая длинная строка, которую можно прочесть, дойдя до состояния* v *из начального (то есть самое длинное слово класса* v*).*

$\text{len}(v) := |\text{longest}(v)|$.

Определение 4 *(Суффиксная ссылка).* $\text{link}(v)$ *— состояние, в котором лежит самый длинный суффикс* $\text{longest}(v)$*, не лежащий в* v*.*

Для начального состояния (класса $[\varepsilon]$*) ссылка не определена.*

Это та же идея, что и в Ахо-Корасик: ссылка ведёт в «следующий по длине суффикс», только там вершины бора, а здесь — классы.

Пример 1 *(*$S = ababa$*).* $R_S(a) = \{baba, ba, \varepsilon\}$

$R_S(aba) = \{ba, \varepsilon\}$

$R_S(ba) = \{ba, \varepsilon\}$

Значит, $aba \underset{S}{\sim} ba$*, а* a *лежит в другом классе:*

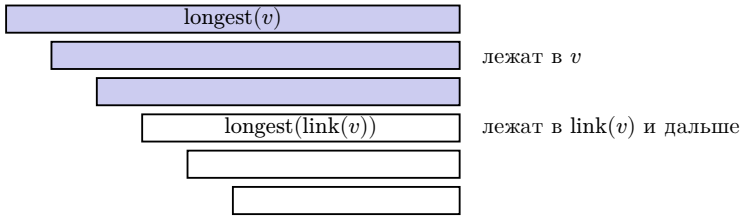
суффиксная ссылка ведёт из класса $\{aba, ba\}$ *в класс* $\{a\}$*:* $\text{link}([aba]) = [a]$

$\text{longest}([a]) = a$, $\text{longest}([aba]) = \text{longest}([ba]) = aba$

Целиком классы здесь такие: $\{\varepsilon\}$, $\{a\}$, $\{ab, b\}$, $\{aba, ba\}$, $\{abab, bab\}$, $\{ababa, baba\}$ *— шесть состояний на строку длины 5.*

Утверждение 2 *(устройство класса). Пусть* v *— вершина автомата. Тогда в* v *лежат* $\text{longest}(v)$ *и несколько её самых длинных суффиксов — подряд по длине, без пропусков.*

Действительно, по утверждению выше все слова v *— суффиксы* $u = \text{longest}(v)$*. Осталось доказать, что пропусков нет. Пусть* $w \in v$ *и* x *— суффикс* u*, который длиннее* w*. Тогда* w *— суффикс* x*, а* x *— суффикс* u*. Если* uz *— суффикс* S*, то и* xz *— суффикс* S*, значит,* $R_S(u) \subseteq R_S(x)$*. Точно так же* $R_S(x) \subseteq R_S(w)$*. Но* $R_S(u) = R_S(w)$*, поэтому все три множества равны, и* $x \in v$*.*



Обратите внимание: в класс попадают *не все* суффиксы $\text{longest}(v)$. В примере $S = ababa$ класс $\{aba, ba\}$ не содержит a: у a есть лишнее вхождение в начале строки, поэтому $baba \in R_S(a)$, а в $R_S(aba)$ этого нет.

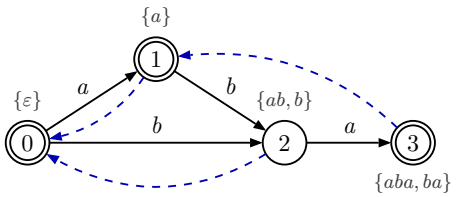
Следствие 1. *Слова класса* v *— это суффиксы* $\text{longest}(v)$ *длин от* $\text{len}(\text{link}(v)) + 1$ *до* $\text{len}(v)$*; их ровно* $\text{len}(v) - \text{len}(\text{link}(v))$ *штук. А цепочка* $v, \text{link}(v), \text{link}(\text{link}(v)), \ldots$ *перебирает* все *суффиксы* $\text{longest}(v)$ *по*

убыванию длины — вплоть до пустого в начальном состоянии. Это аналог леммы о цепочке ссылок из прошлой лекции, дальше этот путь будем называть суффиксным путём.

Пример 2 *($S = aba$). Подстроки: $\varepsilon, a, b, ab, ba, aba$. Считаем правые контексты:*

- $R_S(\varepsilon) = \{aba, ba, a, \varepsilon\}$;
- $R_S(a) = \{ba, \varepsilon\}$;
- $R_S(b) = R_S(ab) = \{a\}$;
- $R_S(ba) = R_S(aba) = \{\varepsilon\}$.

Получаем четыре класса: $\{\varepsilon\}$, $\{a\}$, $\{ab, b\}$, $\{aba, ba\}$.



Сплошные рёбра — переходы автомата, пунктир — суффиксные ссылки, двойной линией обведены терминальные состояния. Сравните с примером: из $\{a\}$ до терминалов ведут пути ba и ε — это ровно $R_S(a)$; из $\{ab, b\}$ — только путь a. Терминальные состояния — это $[aba]$, $[a]$ и $[\varepsilon]$, то есть суффиксный путь из состояния всей строки: он перебирает все суффиксы S.

Теорема 1 *(критерий longest). Пусть u — подстрока S. Тогда $u = \text{longest}([u]_S)$ тогда и только тогда, когда*

- *либо u — префикс S,*
- *либо существуют различные буквы $a \neq b$ такие, что au и bu — подстроки S.*

Иными словами, u нельзя продлить влево, не потеряв вхождение: либо у какого-то вхождения слева ничего нет, либо слева стоят разные буквы.

Доказательство.

$\Leftarrow$ Пусть u — не $\text{longest}([u])$. Тогда $\text{longest}([u]) = \gamma u$, где $|\gamma| \geq 1$. Посмотрим на любое вхождение u в S, пусть после него идёт кусок w до конца строки. Тогда $w \in R_S(u) = R_S(\gamma u)$, то есть $\gamma u w$ — тоже суффикс S, и перед этим вхождением u стоит γ. Значит, перед *каждым* вхождением u что-то стоит (поэтому u — не префикс), и это всегда последняя буква γ (поэтому двух разных букв слева нет).

$\Rightarrow$ Пусть $u = \text{longest}([u])$, но u — не префикс S. Посмотрим на буквы перед всеми вхождениями u (они есть, раз u не префикс). Если среди них нет двух различных, то все они равны какой-то d. Но тогда $R_S(du) = R_S(u)$: включение $\subseteq$ есть всегда, а обратно — если uw суффикс S, то перед этим вхождением u стоит d, и duw — тоже суффикс. Получили $du \underset{S}{\sim} u$ и $|du| > |u| = \text{len}([u])$ — противоречие с тем, что u самое длинное.

Пример 3 *($S = abb$). Не стоит думать, что longest бывают только у префиксов. Классы здесь: $\{\varepsilon\}$, $\{a\}$, $\{ab\}$, $\{b\}$, $\{abb, bb\}$. У класса $\{b\}$ самое длинное слово b — не префикс S. Но критерий выполнен: ab и bb — подстроки, перед b встречаются разные буквы. А вот bb не longest: единственное вхождение, и перед ним a.*

Сам автомат для abb нарисован ниже, в примере с поэтапным построением.

Следствие 2 *(aka чеховское ружьё). Что было longest, то навсегда им и будет: если $u = \text{longest}([u]_S)$, то и $u = \text{longest}([u]_{Sc})$ для любой буквы c.*

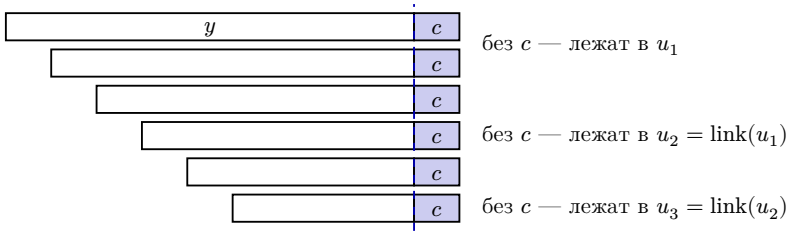
Действительно, дописывание буквы в конец не отнимает у и ни вхождений, ни букв перед ними: префикс S остаётся префиксом Sc, а подстроки au и bu никуда не деваются.

Утверждение 3 *(рёбра в одну вершину). Пусть $u_1 \to v, u_2 \to v, ..., u_k \to v$ — все рёбра, входящие в v, причём $\text{len}(u_i) > \text{len}(u_{i+1})$. Тогда*
1. *все эти переходы идут по одной и той же букве c;*
2. $\text{link}(u_i) = u_{i+1}$.

Доказательство.

1. Если есть ребро $(u_i, c) \to v$, то для любого $x \in u_i$ слово xc лежит в v. Но все слова v — суффиксы $\text{longest}(v)$, а значит, заканчиваются одной и той же буквой — последней буквой $\text{longest}(v)$. Это и есть c.

2. Пусть $\text{longest}(v) = yc$. По утверждению об устройстве класса в v лежат суффиксы yc длин из отрезка $(\text{len}(\text{link}(v)), \text{len}(v)]$. Откусим от них последнюю букву — получим суффиксы y, тоже *подряд* по длине. Каждый такой суффикс x лежит в каком-то u_i (ведь $xc \in v$), и наоборот, любое слово u_i даёт при дописывании c слово из v. Выходит, отрезок подряд идущих суффиксов y разрезан на куски-классы $u_1, u_2, ..., u_k$ (по убыванию длины), а по определению суффиксной ссылки следующий кусок после u_i — это $\text{link}(u_i)$.



На картинке — все слова класса v (одна строка — одно слово). Если убрать у них общую последнюю букву c, они распадутся на куски, и эти куски — подряд идущие вершины суффиксного пути. То есть **все рёбра в одну вершину выходят из отрезка суффиксного пути**. Это главное наблюдение для алгоритма построения.

1.3 Что меняется при дописывании буквы

Строить автомат будем, как и префикс-функцию, по одной букве: пусть автомат для S уже есть, а мы дописали в конец букву c. Какие классы при этом появятся?

Утверждение 1 *(новые состояния). При дописывании буквы c к строке S появляется не более двух новых состояний:*
1. *состояние $[Sc]$ с $\text{longest} = Sc$ — появится обязательно;*
2. *состояние $[T]$, где T — самый длинный суффикс Sc, который является подстрокой S, — может появиться, а может и нет (появится, если T не было longest в S).*

Доказательство. Состояние однозначно задаётся своим longest, поэтому новые состояния — это слова, которые *стали* longest (по чеховскому ружью старые longest никуда не денутся). Посмотрим, кто это может быть, по критерию longest.

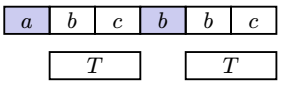
- **Новый префикс** один — сама Sc. Она заведомо стала longest: это префикс Sc.
- **Новые две буквы слева.** Новые вхождения появились только у суффиксов Sc. Суффиксы, которые длиннее T, в S не встречались — у них вхождение одно, и все они лежат в классе $[Sc]$ (правый контекст у всех — $\{\varepsilon\}$). Longest среди них только сама Sc: у любого более

короткого из них вхождение одно, перед ним стоит буква, и это не префикс — критерий не выполнен. Остаются суффиксы длины не больше $|T|$. Для собственного суффикса x строки T новое вхождение в конце предварено той же буквой, что и внутри T, а T в S уже встречалась, — значит, эта буква перед x уже была, ничего нового. И только у самой T буква d перед новым вхождением *точно* новая: если бы dT встречалось в S, то dT было бы более длинным суффиксом Sc, встречающимся в S. Поэтому в Sc слово T обязательно longest: либо оно было префиксом, либо перед его вхождением в S стоит буква, отличная от d (случай $T = \varepsilon$ тривиален — пустое слово всегда longest начального состояния).

Значит, кроме $[Sc]$ новым может стать только $[T]$: если T уже было longest в S, нового состояния нет, иначе — появится.

Пример 1 *($S = abcbb$, дописываем c). $Sc = abcbbc$. Суффиксы: $c, bc, bbc, \ldots$; из них в S встречаются только c и bc, так что $T = bc$.*

В S класс $[bc]$ был $\{abc, bc, c\}$ с longest abc: перед единственным вхождением bc стояла a. После дописывания у bc появилось вхождение в конце, и перед ним стоит b. По критерию bc стало longest, класс раскололся на $\{abc\}$ и $\{bc, c\}$. Итого новых состояний два: $[Sc]$ и $[bc]$ (было 7 состояний, стало 9).

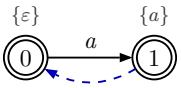


разные буквы перед $T = bc \Rightarrow T$ стало longest

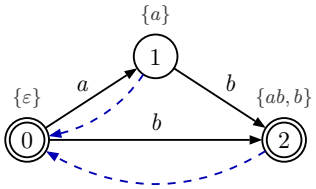
Утверждение 2 *(ссылка нового состояния).* $\mathrm{link}([Sc]_{Sc}) = [T]_{Sc}$, *где T из утверждения выше.*

Действительно, из доказательства видно, что в $[Sc]$ лежат ровно суффиксы Sc, которые длиннее T. Самый длинный суффикс, который туда уже не попал, — это T. И T действительно лежит в другом классе, чем Sc: T встречается ещё и внутри S, поэтому в $R_{Sc}(T)$ есть непустое слово, а у $[Sc]$ правый контекст — только $\{\varepsilon\}$.

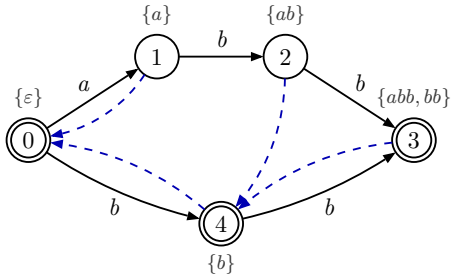
Пример 2 *(поэтапное построение для abb). Посмотрим, как растёт автомат, когда мы дописываем по букве. Номера состояний — в порядке их появления.*



$S = a$. Два класса: $\{\varepsilon\}$ и $\{a\}$. $T = \varepsilon$ (буквы a раньше не было), $\mathrm{link}(1) = 0$.



$S = ab$. Буквы b раньше не было, поэтому $T = \varepsilon$ — старый класс, новое состояние одно: $[ab] = \{ab, b\}$. В него ведут рёбра по b из всех состояний суффиксного пути $1 \to 0$ — это ровно утверждение про рёбра в одну вершину.



$S = abb$. Теперь $T = b$: оно встречалось в ab (с буквой a перед ним), а теперь появилось с буквой b перед ним. Значит, b стало longest, и класс $\{ab, b\}$ раскололся на $\{ab\}$ и $\{b\}$ — появились сразу два новых состояния: $3 = [abb]$ и $4 = [b]$ (как в примере с $S = abb$ выше). И по утверждению о ссылке $\text{link}(3) = 4 = [T]$.

1.4 Размер автомата

Утверждение 1. *При $|S| \geq 2$ число состояний в суффиксном автомате не превосходит $2|S| - 1$.*

Действительно, индукция по длине S. Для строки из двух букв состояний три ($\{\varepsilon\}$, $\{a\}$, $\{ab, b\}$ для ab или $\{\varepsilon\}$, $\{a\}$, $\{aa\}$ для aa). А каждая следующая буква по утверждению выше добавляет не больше двух состояний.

Оценка точная: у строки $abb...b$ ровно $2|S| - 1$ состояний — каждая новая b расщепляет класс, как в примере с abb.

Теорема 1 *(б/д). При $n \geq 3$ в суффиксном автомате строки длины n число рёбер не превосходит $3n - 4$.*

Итого автомат занимает $O(|S|)$ памяти — вместо $O(|S|^2)$ у бора всех суффиксов. Осталось научиться быстро его строить.

1.5 Линейный алгоритм

Алгоритм итеративный: храним автомат для S и состояние $\text{last} = [S]$ (класс всей строки), дописываем букву c и перестраиваем. Что может поменяться:

1. появляются рёбра в новое состояние $[Sc]$ (а рёбер *из* него нет: Sc — самая длинная подстрока, продолжать её некуда);
2. если появилось новое состояние $[T]$ — появляются рёбра в него и из него, а какие-то старые рёбра меняют конец.

Все эти изменения удобно разобрать, идя по суффиксному пути из last — по нему, как мы знаем, лежат все суффиксы S по убыванию длины.

Рёбра в $[Sc]$. Какие состояния должны вести по c в $[Sc]$? Те, в которых лежит суффикс x строки S, такой что xc — *не* подстрока S (тогда xc длиннее T и лежит в $[Sc]$). Идём по суффиксному пути из last: пока у состояния нет ребра по c, проводим ребро в $[Sc]$ и переходим по ссылке. Как только встретилось состояние p, из которого ребро по c уже есть, — останавливаемся: для него и всех более коротких суффиксов xc уже встречалось в S. (Это согласуется с утверждением о рёбрах в одну вершину: в $[Sc]$ ведёт отрезок суффиксного пути, начинающийся в $[S]$.)

Случай 1. Если мы дошли до начального состояния q_0 и ребра по c нет и у него, значит, буквы c в S не было вовсе. Тогда $T = \varepsilon$ и $\text{link}([Sc]) = q_0$. Готово.

Случай 2. Пусть мы остановились в p и есть ребро $(p, c) \to q$. По построению $\text{longest}(p)$ — самый длинный суффикс S, который продолжается буквой c внутри S, то есть

$$T = \text{longest}(p)\, c, \quad |T| = \text{len}(p) + 1$$

T лежит в q. Новое состояние появляется, если $T \neq \text{longest}(q)$, то есть критерий расщепления:

$$\text{len}(q) > \text{len}(p) + 1$$

Если же $\text{len}(q) = \text{len}(p) + 1$, то T уже было longest, класс q не меняется, и достаточно поставить ссылку $\text{link}([Sc]) = q$. На этом всё.

Случай 3. Теперь $\text{len}(q) > \text{len}(p) + 1$: класс q раскалывается. Появляется новое состояние clone с $\text{longest}(\text{clone}) = T$, в него уходят T и её суффиксы из q, а в q остаются слова длиннее T.

- **Рёбра в clone.** Раньше все рёбра по c в q шли из отрезка суффиксного пути. Те из них, что идут из p и дальше по пути (из более коротких суффиксов), теперь должны вести в clone: слова xc для них не длиннее T. Прыгаем по ссылкам из p, пока ребро по c ведёт в q, и перенаправляем его в clone.
- **Рёбра из clone.** Слова clone отличаются от слов q только тем, что они ещё и суффиксы Sc: $R_{Sc}(\text{clone}) = R_{Sc}(q) \sqcup \{\varepsilon\}$. Непустые продолжения те же, поэтому переходы из clone — копия переходов из q.
- **Ссылки.** $\text{link}([Sc]) = \text{clone}$ (это $[T]$), $\text{link}(\text{clone}) = \text{link}(q)$ (старая), $\text{link}(q) = \text{clone}$, и $\text{len}(\text{clone}) = \text{len}(p) + 1$.

Алгоритм 1: Построение суффиксного автомата

Начинаем с одного состояния q_0 (класс $[\varepsilon]$), $\text{last} = q_0$. Для каждой следующей буквы c:

1. Заводим состояние $\text{cur} = [Sc]$, $\text{len}(\text{cur}) = \text{len}(\text{last}) + 1$.
2. Идём $p = \text{last}, \text{link}(\text{last}), ...$, пока у p нет ребра по c, и проводим рёбра $(p, c) \to \text{cur}$.
3. Если дошли до конца пути и ребра так и не нашлось, то $\text{link}(\text{cur}) = q_0$.
4. Иначе пусть $q = \text{to}(p, c)$. Если $\text{len}(q) = \text{len}(p) + 1$, то $\text{link}(\text{cur}) = q$.
5. Иначе создаём clone: рёбра и ссылка — как у q, $\text{len}(\text{clone}) = \text{len}(p) + 1$. Идём от p по ссылкам, пока ребро по c ведёт в q, и перенаправляем его в clone. Затем $\text{link}(q) = \text{link}(\text{cur}) = \text{clone}$.
6. $\text{last} = \text{cur}$.

После того как все буквы дописаны, помечаем терминальными все состояния суффиксного пути из last — именно в них лежат суффиксы S.

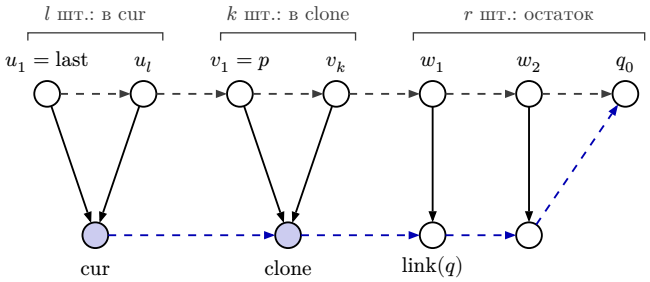
Время работы

Шаги 1, 3, 4, 6 — это $O(1)$ на букву. Цикл на шаге 2 каждую итерацию проводит *новое* ребро, а рёбра никогда не удаляются (на шаге 5 они только меняют конец). Рёбер в итоге $O(|S|)$, значит, и итераций этого цикла суммарно $O(|S|)$. Копирование рёбер в clone — $O(|\Sigma|)$, и клонов не больше $|S|$.

На первый взгляд кажется, что цикл перенаправления на шаге 5 может каждый раз проходить длинный кусок суффиксного пути, и суммарно выйдет не $O(|S|)$. Посмотрим на длину m суффиксного пути из last (число вершин на нём). До шага 5 он выглядит так:

$$[u_1, ..., u_l,\ v_1, ..., v_k,\ w_1, ..., w_r]$$

где из u_i мы провели рёбра в cur ($l \geq 1$: из last ребра по c быть не может, иначе Sc была бы подстрокой S), у $v_1 = p, ..., v_k$ ребро по c вело в q и было перенаправлено в clone, а w_j — остаток пути до q_0.



Серым пунктиром — старый суффиксный путь, синим — новый, сплошные стрелки — рёбра по c. Новый путь короткий: в нём cur, clone, а дальше каждая вершина, кроме q_0, «оплачена» своей вершиной w_j сверху — так что все u_i и v_i из длины пропадают.

Новый суффиксный путь — cur $\to$ clone $\to$ link(q) $\to$ .... Каждая вершина пути из link(q), кроме q_0, получает ребро по c из своего w_j (её longest — это zc, где z — суффикс S короче слов из v_i, то есть z лежит в каком-то w_j), причём из разных w_j — ведь по одной букве из состояния ведёт одно ребро. Поэтому путь из link(q) не длиннее $r + 1$, и новая длина

$$m' \leq 2 + r + 1 = m + 3 - l - k \leq m + 2 - k$$

В случаях 1 и 2 то же неравенство верно с $k = 0$. То есть за одну букву длина суффиксного пути растёт не больше чем на 2, а каждое перенаправленное ребро уменьшает её на 1. Длина неотрицательна, значит, всего перенаправлений не больше $2|S|$. Итого — $O(|S|)$ при константном алфавите.

Примечание. Вернёмся к поэтапному построению abb и посмотрим, какие случаи там сработали.

- $\varepsilon \to a$ и $a \to ab$: буква новая, мы проходим весь суффиксный путь до q_0, проводя рёбра, — **случай 1**.
- $ab \to abb$: из last $= 2$ ребра по b нет — проводим его в 3, переходим в q_0. Из q_0 ребро по b есть, оно ведёт в $q = [ab, b]$, и len(q) $= 2 >$ len(q_0) $+ 1 = 1$ — **случай 3**: создаём clone $= 4 = [b]$ с копией ребра $(4, b) \to 3$, перенаправляем (q_0, b) в 4, ставим link(2) $=$ link(3) $= 4$.
- Для **случая 2** достроим $ab \to aba$: из last $= [ab, b]$ проводим ребро по a в новое состояние, переходим в q_0, у него ребро по a ведёт в $[a]$, и len($[a]$) $= 1 =$ len(q_0) $+ 1$. Значит, link($[aba]$) $= [a]$, и никакого расщепления — получается ровно автомат для aba с картинки выше.

Реализация (C++): суффиксный автомат

Имена переменных — как в алгоритме выше: автомат строится в конструкторе, по одной букве за раз.

```
class SuffixAutomaton {
 public:
  SuffixAutomaton(const std::string& text) {
    states_.push_back(State());  // states_[0] — начальное состояние q0
    for (char symbol : text) {
      add_letter(symbol - 'a');
    }
  }

 private:
  struct State {
    int len = 0;
    int link = -1;
    int next[26];  // переходы по буквам, -1 — ребра нет

    State() {
      for (int letter = 0; letter < 26; ++letter) {
        next[letter] = -1;
      }
    }
  };

  void add_letter(int c) {
    int cur = states_.size();                       // шаг 1
    states_.push_back(State());
    states_[cur].len = states_[last_].len + 1;
    int p = last_;
    while (p != -1 && states_[p].next[c] == -1) {   // шаг 2
      states_[p].next[c] = cur;
      p = states_[p].link;
    }
    if (p == -1) {                                  // шаг 3
      states_[cur].link = 0;
    } else {
      int q = states_[p].next[c];
      if (states_[p].len + 1 == states_[q].len) {   // шаг 4
        states_[cur].link = q;
      } else {                                      // шаг 5
        int clone = states_.size();
        states_.push_back(states_[q]);              // рёбра и link — как у q
        states_[clone].len = states_[p].len + 1;
        while (p != -1 && states_[p].next[c] == q) {
          states_[p].next[c] = clone;
          p = states_[p].link;
        }
        states_[q].link = clone;
        states_[cur].link = clone;
      }
    }
```

```
    last_ = cur;                                    // шаг 6
  }

  std::vector<State> states_;
  int last_ = 0;
};
```

Терминальные состояния, если они нужны, — это `last_`, `states_[last_].link` и так далее до `-1`.

Примечание. Если коротко, суффиксный автомат держится на таких фактах:

1. Состояние — это класс слов с одинаковым правым контекстом (Майхилл-Нероуд), и в нём лежат несколько подряд идущих по длине суффиксов одного слова — longest.
2. Слово является longest, если оно префикс или перед ним встречаются разные буквы; раз став longest, слово им и остаётся.
3. Поэтому при дописывании буквы появляется не больше двух состояний — $[Sc]$ и, может быть, $[T]$, — и автомат имеет размер $O(|S|)$.
4. Все рёбра в одну вершину выходят из отрезка суффиксного пути, поэтому всё перестроение делается прыжками по ссылкам из last.